Introduction to Programming and Data Structures

Binary Trees

Malay Bhattacharyya

Associate Professor

MIU, CAIML, TIH
Indian Statistical Institute, Kolkata

October, 2023

1 Basics

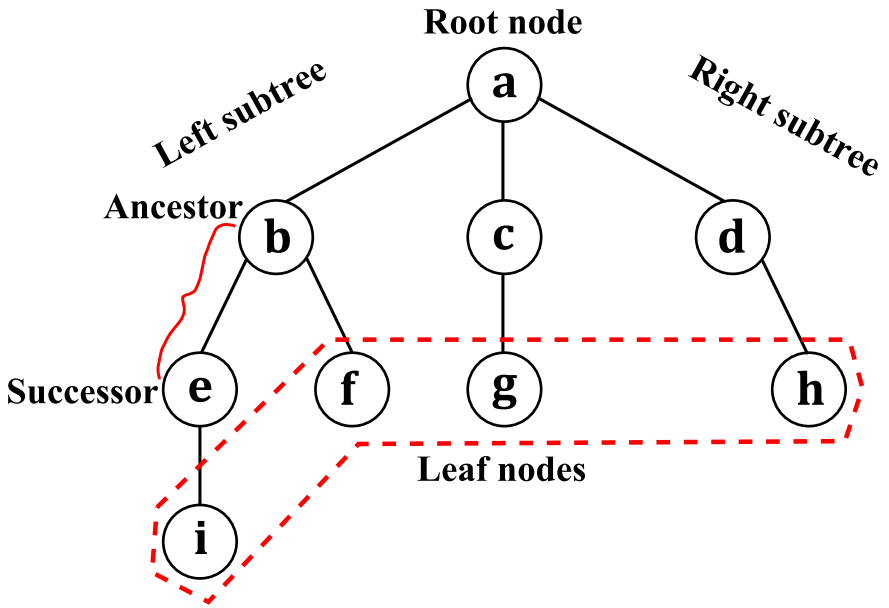
2 Implementation

3 Traversal

4 Problems

Basics of a tree

A tree is recursively defined as a set of one or more nodes where a single node is designated as the root of the tree and all the remaining nodes can be partitioned into subtrees of the root.



Note: Tree is a non-linear data structure (a data item can be linked to more than two data items).

Types of trees

Trees can be of different types based on their use as data structures. Some of these are listed below.

- **Forests:** Disjoint union of trees.
- **Binary trees:** Trees having at most two subtrees per node.
- **Binary search trees:** Binary trees in which the nodes are arranged in an order (based on the data items that they keep).
- **Expression trees:** Trees that are used to store algebraic expressions.
- **Tournament trees:** Trees that are used to represent players using the leaf nodes and winners using the remaining nodes.

Note: Trees are data structures that are mainly used to store data items (labels or keys on the nodes) hierarchically.

Binary trees

A binary tree is a special type of tree where each node has a degree equal to or less than two which means each node should have at most two children.

Definition (Binary Tree)

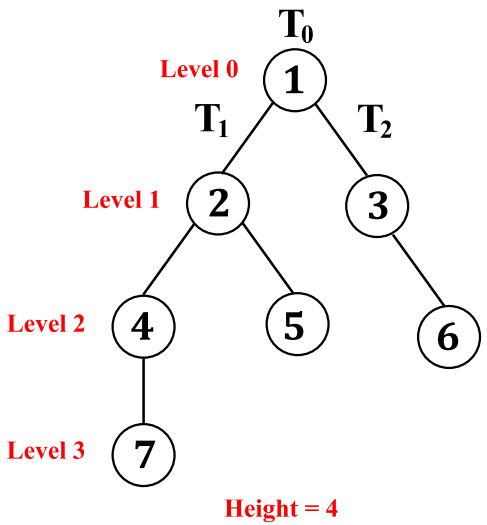
A binary tree T over a domain D is either an empty set (empty binary tree) or a recursively-defined triplet $\langle T_0, T_1, T_2 \rangle$ such that

- T_0 is a node over D (root), and
- T_1 and T_2 are binary trees over D (left and right subtree, respectively).

If there are n nodes in a binary search tree, maximum height of the binary search tree is n-1 and minimum height is ceil(log2(n+1))-1.

If binary search tree has height h, minimum number of nodes is h+1 (in case of left skewed and right skewed binary search tree).

If binary search tree has height h, maximum number of nodes will be when all levels are completely full. Total number of nodes will be 2^(h+1)-1.



Terminologies in binary trees

- **Left child:** Left successor node of a node.
- **Right child:** Right successor node of a node.
- **Parent:** Ancestor node of a set of children.
- **Sibling:** Nodes that are at the same level and share the same parent.
- **Degree of a node:** Number of children of a node.

Note: Every node other than the root node has a parent.

Depth: Depth of a node is the number of edges from root to that node.

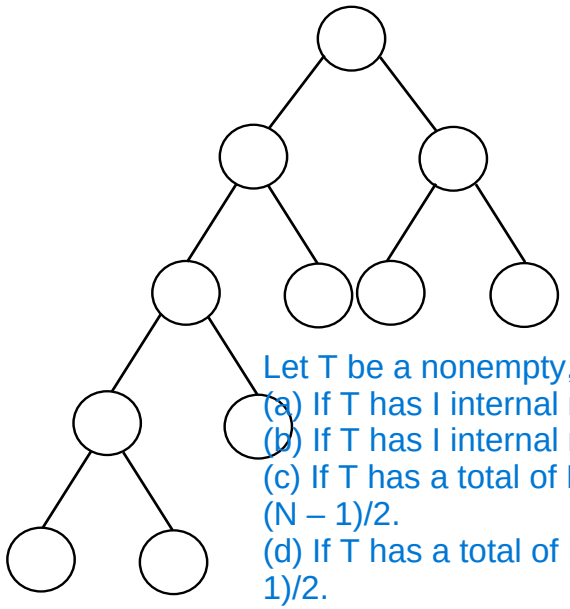
Height: Height of a node is the number of edges from that node to the deepest leaf.

Full binary trees

strict binary tree is a binary tree having all of its nodes with a degree of 2 or 0

Definition (Full Binary Tree)

A binary tree is said to be full binary tree (or 2-tree) if each node in the tree has either no child or exactly two children.



Let T be a nonempty, full binary tree Then:
(a) If T has I internal nodes, the number of leaves is L = I + 1.
(b) If T has I internal nodes, the total number of nodes is N = 2I + 1.
(c) If T has a total of N nodes, the number of internal nodes is I = (N – 1)/2.
(d) If T has a total of N nodes, the number of leaves is L = (N + 1)/2.
(e) If T has L leaves, the total number of nodes is N = 2L – 1.
(f) If T has L leaves, the number of internal nodes is I = L – 1.

Note: An extended binary tree becomes a full binary tree.

A perfect binary tree has all its internal nodes with degree strictly 2 and has all its leaf nodes on the same level.

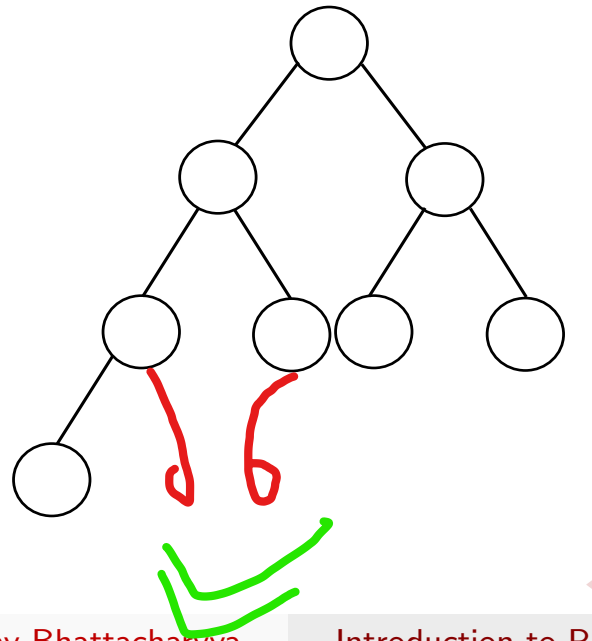
Complete binary trees

A complete binary tree has all its levels completely filled except possibly the last level. And if the last level is not completely filled then the last level's keys must be all left-aligned

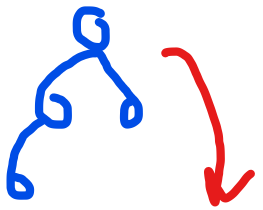
Definition (Complete Binary Tree)

A binary tree is said to be complete binary tree if it satisfies both of the following:

- Every level, except possibly the last, is completely filled.
- All nodes appear as far left as possible.



Complete binary trees



Some properties:

height(h) <=> levels(h+1) (2^0 + 2^1 + 2^2 + ... + 2^h) + 1

1. A complete binary tree of height h comprises at least 2^h and at most $2^{h+1} - 1$ nodes.



(2^0 + 2^1 + 2^2 + ... + 2^h + 2^h+1)

2. The height of a complete binary tree having exactly n nodes is $\lceil \log_2(n+1) \rceil$.
3. The maximum possible number of parents with absent children in a complete binary tree having n nodes is $(n+1)$.
4. The number of internal nodes in a complete binary tree having n nodes is $\lfloor n/2 \rfloor$.

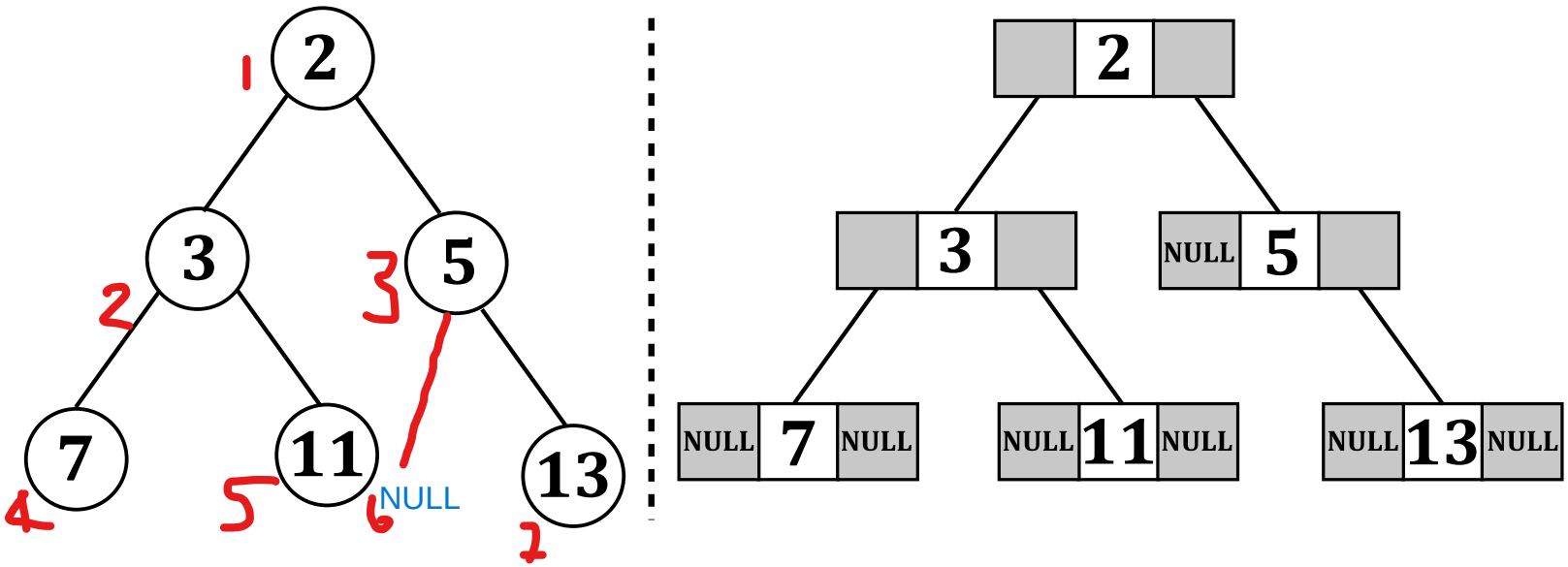
Internal node: Nodes with at least one child

N is the number of nodes, h is the height of a complete binary tree:
2**h <= N < 2**(h+1)
=> h <= ln2(N) < h + 1
=> h = floor(ln2(N))
ceiling(log(N+1)) -1 is same as floor(ln2(N))

Conventional implementation – An example

We first assume the remaining vacant places to be NULL.
Traverse each level starting from the root node and from left to right and mark them with the indices these nodes would belong to.

use doubly-linked lists



Conventional implementation of binary trees

```
root = {'Data': 2, 'Left': None, 'Right': None}
root['Left'] = {'Data': 3, 'Left': None, 'Right': None}
root['Right'] = {'Data': 5, 'Left': None, 'Right': None}
l = root['Left']
l['Left'] = {'Data': 7, 'Left': None, 'Right': None}
r = root['Right']
l['Right'] = {'Data': 11, 'Left': None, 'Right': None}
r = root['Right']
r['Right'] = {'Data': 13, 'Left': None, 'Right': None}
```

Note: The NULL value is implemented as None here.

Alternative implementation – An example

Initially: root = `NULL`

	Data	Left	Right
free → 0	–	1	NULL
1	–	2	NULL
2	–	3	NULL
3	–	4	NULL
⋮	⋮	⋮	⋮
$n-1$	–	NULL	NULL

Note: The `NULL` value is implemented as `None` here.

Alternative implementation – An example

Finally:

	INDEX	Data	Left	Right
			Index values	
root →	0	2	1	2
	1	3	3	4
	2	5	NULL	5
	3	7	NULL	NULL
	4	11	NULL	NULL
	5	13	NULL	NULL
free →	6	–	7	NULL
	⋮	⋮	⋮	⋮
	$n-1$	–	NULL	NULL

Note: The `NULL` value is implemented as `None` here.

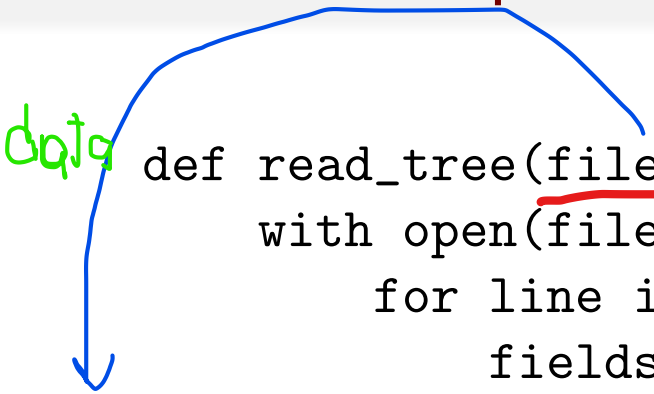
Alternative implementation of binary trees

```
def print_tree(BT):
    print(len(BT))
    for i in range(N):
        print(BT[i]['Data'], BT[i]['Left'], BT[i]['Right'])
    return
```

Alternative implementation of binary trees

map(fun, iter)

Returns a list of the results after applying the given function to each item of a given iterable (list, tuple etc.)




```
def read_tree(filename):
    with open(filename) as f:
        for line in f:
            fields = list(map(int, line.split()))
            if len(fields) == 1: # Read the first line
                N = int(fields[0])
                BT = [ {'Data': None, 'Left': None,
                'Right': None}
                for i in range(N) ]
                index = 0
            else:
                BT[index]['Data'] = fields[0]
                BT[index]['Left'] = fields[1]
                BT[index]['Right'] = fields[2]
                index += 1
    return tree
```

(which is total number of nodes)

Sequential implementation of binary trees

- One time memory allocation for a one-dimensional array.
- The left and right child of the node at index i are at index $2i$ and $2i + 1$, respectively.
- It might involve a lot of unused allocated memories.

```
BT = [2, 3, 5, 7, 11, None, 13]
print(len(BT))
for i in range(BT):
    ...
```

Sequential implementation – An example

		Data
	0	NULL
root →	1	2
	2	3
	3	5
	4	7
	5	11
	6	NULL
	7	13

parent (at index i)

no left child (at index $2i$)

right child (at index $2i + 1$)

Note: The `NULL` value is implemented as `None` here.

Traversal of a binary tree

- Pre-order traversal
- In-order traversal
- Post-order traversal
- Level-order traversal

Pre-order traversal

Perform the following operations recursively at each node:

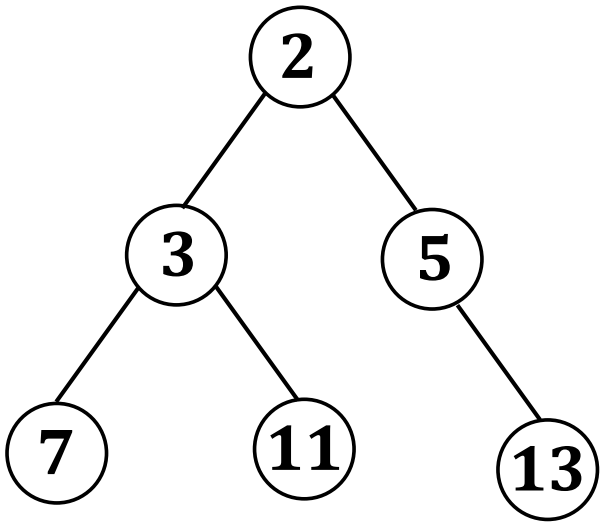
1. Visit the root node.
2. Traverse the left subtree.
3. Traverse the right subtree.

Pre-order traversal

root -> left -> right

Perform the following operations recursively at each node:

1. Visit the root node.
2. Traverse the left subtree.
3. Traverse the right subtree.



Order of visited nodes: 2, 3, 7, 11, 5, 13

Pre-order traversal

```
def preOrder(BT):
    if BT == None:
        return
    print(BT['Data'], end = ' ')
    preOrder(BT['Left'])
    preOrder(BT['Right'])
```

In-order traversal

Perform the following operations recursively at each node:

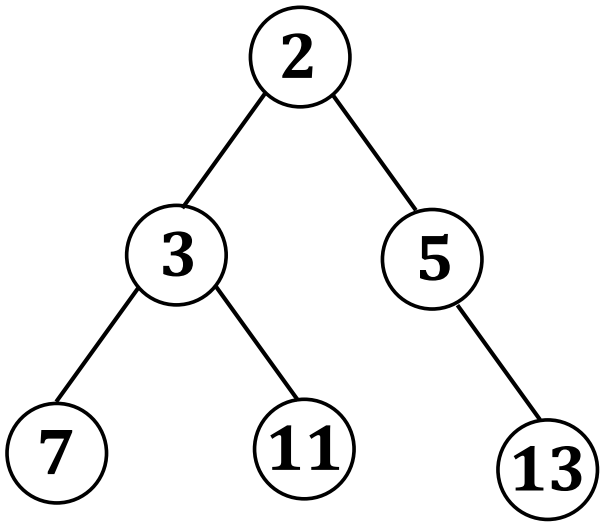
1. Traverse the left subtree.
2. Visit the root node.
3. Traverse the right subtree.

In-order traversal

left -> root -> right

Perform the following operations recursively at each node:

1. Traverse the left subtree.
2. Visit the root node.
3. Traverse the right subtree.



Order of visited nodes: 7, 3, 11, 2, 5, 13

In-order traversal

```
def inOrder(BT):
    if BT == None:
        return
    inOrder(BT['Left'])
    print(BT['Data'], end = ' ')
    inOrder(BT['Right'])
```

Post-order traversal

Perform the following operations recursively at each node:

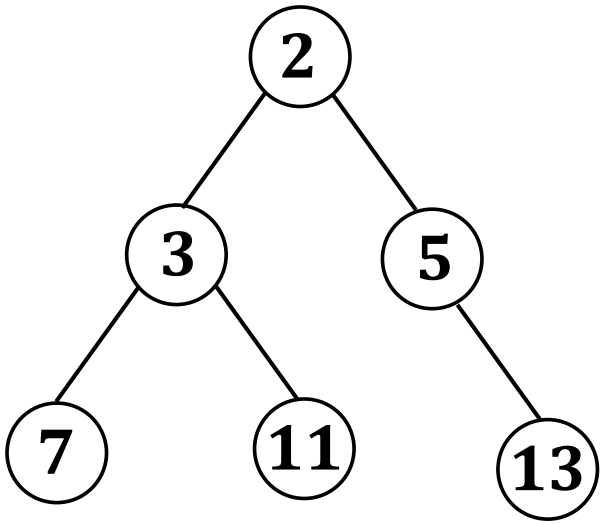
1. Traverse the left subtree.
2. Traverse the right subtree.
3. Visit the root node.

Post-order traversal

left -> right -> root

Perform the following operations recursively at each node:

1. Traverse the left subtree.
2. Traverse the right subtree.
3. Visit the root node.



Order of visited nodes: 7, 11, 3, 13, 5, 2

Post-order traversal

```
def postOrder(BT):
    if BT == None:
        return
    postOrder(BT['Left'])
    postOrder(BT['Right'])
    print(BT['Data'], end = ' ')
```

Level-order traversal

Perform the following operations recursively at each node:

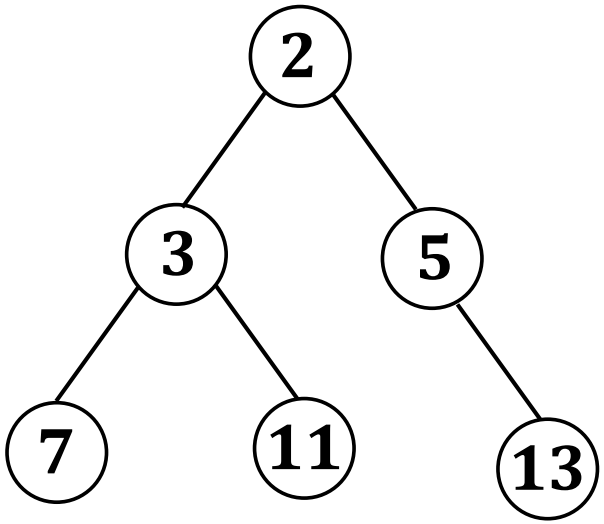
1. Visit the root node.
2. Traverse all the subtrees from left to right at the next level.

Level-order traversal

from left to right, each level

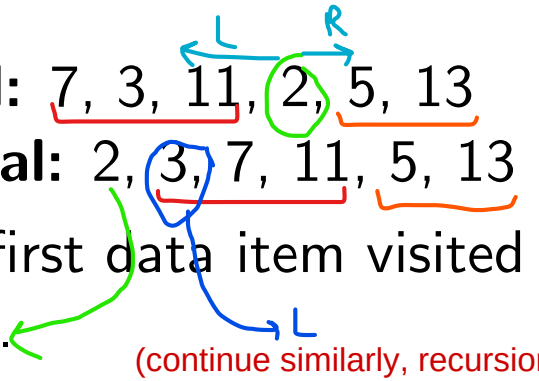
Perform the following operations recursively at each node:

1. Visit the root node.
2. Traverse all the subtrees from left to right at the next level.



Order of visited nodes: 2, 3, 5, 7, 11, 13

Constructing binary trees from traversal results



In-order traversal: 7, 3, 11, 2, 5, 13 left , node , right

Pre-order traversal: 2, 3, 7, 11, 5, 13 node , left , right

1. Consider the first data item visited in pre-order traversal as the root node.
2. Data items on the left and right side of the root node in the in-order traversal sequence form the left and right subtree of the root node, respectively.
3. Recursively select each data item from pre-order traversal sequence and create its left and right subtrees from the in-order traversal sequence.

Constructing binary trees from traversal results – An example

In-order traversal: 7, 3, 11, 2, 5, 13
Pre-order traversal: 2, 3, 7, 11, 5, 13

- **Step 1:** Data item '2' is the root node (first in pre-order).
- **Step 2:** Left and right subtree of '2' comprises $\{7, 3, 11\}$ and $\{5, 13\}$, respectively (items on the left and right of '2' in in-order).
- **Step 3:** '3' and '5' are the root nodes of the left and right subtree of '2' (first of those data items in pre-order).
- **Step 4:** Left and right subtree of '3' comprises $\{7\}$ and $\{11\}$, respectively (items on the left and right of '3' in in-order).
- **Step 5:** Left and right subtree of '5' comprises $\emptyset$ and $\{13\}$, respectively (items on the left and right of '5' in in-order).
- **Step 6:** Subtrees of other nodes are $\emptyset$.

Problems

1. Suppose a complete binary tree is represented with an array. In this array, the root node is placed at index 0. The left and right child of any arbitrary node at the index i is placed at the index $2i + 1$ and $2(i + 1)$, respectively. Write a program to construct the tree hierarchically by taking its data items as an array from the user and print the pre-order traversal result.
2. Given a binary tree with integer-valued nodes, and a *target* value, determine whether there exists a root-to-leaf path in the tree such that the sum of all node values along that path equals the target. Modify your program to consider all *paths*, not just root-to-leaf paths.